

PLANNING DOCUMENT · AUGUST 2026

Broadening BMFinder beyond Arma 3

A scope assessment for v2.25.0, plus a review of the interface as it stands today.

Who this is for

Someone technical who has not lived inside this codebase. Every term that could be jargon is explained the first time it appears, and nothing assumes you remember how the extension is put together. If a sentence needs you to already know something, that is a fault in the sentence.

What it answers

- Can BMFinder support games other than Arma 3, and what would that take?
- What could go wrong, technically and with the Chrome Web Store?
- What should be built first, so the later work has something to stand on?
- Where is the interface weakest right now, regardless of scope?

The short answer

BMFinder is already most of the way to multi-game. The database stores a game for every server, the page reader already understands any game, and player search never mentioned a game at all.

One line of code is the actual gate. The real work is not the code — it is the listing, the testing, and deciding what the product is for.

1. The words used in this document

Skip this if you already know them. It is here so nothing later has to stop and explain itself.

Slug. A short, lowercase, URL-safe name for something. BattleMetrics identifies each game with one: Arma 3 is **arma3**, and a server page lives at **/servers/arma3/12345**. Broadening scope is, at heart, letting that part of the address be something other than **arma3**.

Extractor. The part of BMFinder that reads a BattleMetrics page and pulls out the useful facts — a server name, who is on it, a player's previous names. It reads the finished page, the way a person would, rather than calling an API.

API. A machine-readable way to ask a website for data directly. BattleMetrics closed theirs to non-paying users in July 2026, which is the entire reason BMFinder is a browser extension instead of a script.

Manifest. The small settings file every extension has. It declares the name, the version, and which permissions the extension wants.

Single purpose. A Chrome Web Store rule: an extension must do one coherent thing. It governs what the extension is for, not how many games it covers. This distinction matters later.

Schema migration. Changing the shape of stored data after people already have data in the old shape. It is the expensive kind of change, because it has to work on every existing user's machine, once, without losing anything.

This project does not need one.

2. What already works, and why that matters

The instinct with a change like this is to assume the game is baked in everywhere and the job is a rewrite. It is worth being precise, because the truth is much better than that.

Searching the whole extension for the word “arma” returns **eight** results outside the tests. Every one of them is a default — a value used when nothing else is known — rather than a restriction.

Three things are already game-neutral

Component	Current behaviour	Change needed
Player search	The address it requests contains no game at all. BattleMetrics searches players across every game it tracks.	None. This already works for every game today.
Page reader	Reads the game out of whatever address it is given, using a pattern that accepts any slug.	None.
Database	Already stores a game against every followed server, and already fills it in from the page.	None. No migration.

What actually pins it to Arma 3

One line, in the dashboard, where a server search is sent:

Item	What it is	Severity
The server search gate	When you search for a server, the dashboard passes the fixed text arma3 instead of asking which game you meant. Server search is the only address that needs a game in it.	This is the whole gate.
Fallback defaults	A handful of places say “use the stored game, or arma3 if we do not have one”. Harmless for existing data, but they would mislabel a server whose game was never captured.	Cosmetic, but should be tightened.

One real bug, already fixed in v2.0.1

The activity export wrote every server link as `/servers/arma3/<id>` regardless of the actual game. Anyone already following a non-Arma server was getting export links that do not resolve.

That is worth knowing because it proves the point: people can already follow servers from other games. The extension just quietly assumed otherwise in one place.

3. Foundations to lay before v2.25.0

These are ordered. Each one makes the next safer.

Foundation 1 — Decide what the product is for

This is a product decision, not an engineering one, and everything else depends on it. There are two coherent answers and they lead to different builds:

- **One game at a time.** The user picks their game once, in settings, and BMFinder behaves as it does today. Simplest to build, simplest to explain, and covers most people, who play one game seriously.
- **Several games at once.** Followed servers span games, and the interface has to show which game each one belongs to. More useful for a minority, and it puts a new column of information on almost every screen.

The second is more work in the interface than in the logic, which is the opposite of what it looks like from the outside. My recommendation is the first, with the data model left capable of the second — which is already true today.

Foundation 2 — A single list of supported games

Right now the word **arma3** is typed into the source in several places. Before adding more games, those should collapse into one list that everything reads from: the search dropdown, the fallbacks, and any future validation. One list, one place to change it.

This is the smallest possible change and the one that makes every later change safe. It should happen even if the scope never broadens.

Foundation 3 — Confirm the reader works on other games

The extractor is written generically, and nothing in it mentions Arma. But “should work” and “tested” are different claims, and this one has never been tested outside Arma 3.

BattleMetrics renders different games from what appears to be the same template, so the likely outcome is that it simply works. The risk is that some games carry extra fields, or omit ones BMFinder expects. This

needs an afternoon with real pages from three or four games, not a guess.

This is the one genuine unknown

Everything else in this document is either verified or a decision. Whether the page reader copes with, say, a Rust or DayZ server page is the single thing that cannot be settled by reading the code.

Do this first. If it works, the rest is straightforward. If it does not, you want to know before designing an interface around it.

Foundation 4 — Decide what happens to existing users

Everyone currently installed has servers stored with the game **arma3**, or with no game at all if they were added early. Nothing breaks either way, but the interface should not suddenly ask a question they have no reason to care about. A sensible default of Arma 3, changeable in settings, keeps the upgrade invisible for anyone who does not want it.

4. What could go wrong

Chrome Web Store risks

Risk	Real?	What to do
Broadening scope breaks the single-purpose rule	No. The rule is about doing one coherent thing. Supporting more games widens the scope of one purpose; it does not add a second. Delete two words from the current statement and it still describes the product.	Rewrite the statement to drop “Arma 3”. No structural change.
Listing many game names looks like keyword spam	Yes — the main one. Listing games to catch search traffic is exactly the pattern the store penalises, and it is a different violation from the one already survived.	Describe the capability generically. Name at most one or two games as examples. Never build a list.
New permissions disable the extension for existing users	No. The permission already covers the whole of battlemetrics.com, every game included.	Nothing. Verify the manifest is unchanged before submitting.
A fresh review after a prior rejection	Mildly. Any listing change triggers a full re-review, and the account has one rejection on record.	Change code and listing in the same submission. Do not ship half.

The trap worth naming twice

Widening the code while the listing still says Arma 3 — or the reverse — recreates exactly the fault that made the earlier permission justifications indefensible: the paperwork describing something the software does not do.

Whichever you change, change both, in one submission.

Technical risks

- **Page differences between games.** Covered above. The one real unknown, and cheap to settle.

- **Server names collide across games.** Two games can each have a “Main Server”. The internal identifier already keeps them apart, but anything the user reads needs the game shown next to it, or the list becomes ambiguous.
- **Search results get noisier.** Player search already spans every game. Today that is invisible because every followed server is Arma. With several games followed, the “on your servers” column starts mixing them, and will want the game shown.
- **More games means more page loads.** Live updates check every followed server one at a time. Someone following servers across four games has a longer cycle. The pacing already handles this correctly, but the interface should be honest about how long a round takes.

Product risks

- **The name stops describing the product.** BMFinder is “BattleMetrics Finder”, which stays accurate. This is a risk that was avoided by luck rather than planning.
- **Diluted positioning.** “For Arma 3 players” is a sharper pitch than “for players of games BattleMetrics tracks”. Broader is not automatically better. The current audience understands instantly why they want this; a general audience may not.

5. The interface as it stands

Independent of scope. These are things worth fixing whether or not the game support ever broadens.

Measured facts

Measure	Value	Reading
Buttons on the dashboard	60	High. Everything lives on one page behind six tabs.
Text inputs and checkboxes	47	High.
Modal dialogs	3	Reasonable.
Dashboard script	3,843 lines	The main maintainability concern in the project.
Stylesheet	1,569 lines	Large but coherent.
Accessibility labels	13 spoken labels, 51 form labels, 2 live regions	Better than most. Genuinely good.

Finding 1 — Internal names no longer match what users see

The tabs are stored in the code as **watch**, **monitor** and **archive**, but they are shown to users as **People**, **Live activity** and **Recent activity**. The vocabulary was deliberately modernised for release; the internals were not brought along.

Nothing is broken. But anyone new reading the code has to hold a translation table in their head, and the risk is that old words leak back into the interface — which has already happened once, in documents that still described a “watchlist”. Worth a mechanical rename while the codebase is still one person's.

Finding 2 — One file is doing far too much

The dashboard script is 3,843 lines and contains rendering for six tabs, three dialogs, the export logic, the import logic, tag management, and the search interface. Everything works, and it is unusually well commented — but it is the file where a careless change is most likely to break something unrelated.

Splitting it per tab would be a day's mechanical work with immediate benefit, and it becomes considerably harder once a second game adds another dimension to every screen.

Finding 3 — Everything has equal visual weight

Sixty buttons on one page is not automatically wrong, but almost all of them are styled at similar prominence. On the People tab a user can Save a person, Refresh details, Import people and Manage tags, all presented as equally likely next actions. In practice one of those is the common case and the others are occasional.

- Promote the primary action per screen, demote the rest to secondary or into an overflow menu.
- Occasional actions — Manage tags, Import people — do not need permanent header space.

Finding 4 — Six tabs is one or two too many

Live activity and **Recent activity** are adjacent, similarly named, and the difference between them (one is now, one is history) is carried almost entirely by the words “live” and “recent”. That is a lot of weight for two adjectives.

- Consider merging them into one **Activity** tab with a now/history switch inside it.
- If they stay separate, the names should differ by more than one word.

Finding 5 — Long pages, no way back to the top

The full-page screenshots taken for the store listing are over 2,000 pixels tall. Once a user has scrolled to the bottom of a long list, the tab bar is far away and there is no shortcut back.

- A sticky tab bar, or a back-to-top control on long views.

Finding 6 — Empty states do not teach

A fresh install shows empty lists. Version 2.0.1 added guidance to the search screen, which is the sharpest case, but the same problem exists elsewhere: an empty **Servers** tab does not explain that following servers is what makes everything else work.

- Every empty state should say what to do next, not merely that it is empty.

6. Recommended order of work

#	Work	Why now	Size
1	Test the page reader against three or four other games	The only genuine unknown. Everything downstream assumes an answer.	Half a day
2	Collapse the game slug into one shared list	Small, safe, and makes every later change cheaper. Worth doing regardless.	An hour
3	Rename internals to match the user-facing words	Cheapest now, while the codebase is one person's and before another dimension is added.	Half a day
4	Split the dashboard script per tab	Gets harder once multi-game touches every screen.	A day

#	Work	Why now	Size
5	Decide: one game at a time, or several at once	A product decision that determines the interface work.	A conversation
6	Build the game selector and the interface around it	The actual feature.	A day or two
7	Rewrite the listing and submit together	Code and paperwork must move in one step.	Half a day

If only one thing gets done

Test the reader against another game. It is half a day, it settles the only question this document cannot answer, and every other decision here is waiting on it.

A note on the version number

v2.25.0 is a reasonable marker for this work. Under semantic versioning — the convention where the three numbers mean **breaking.feature.fix** — broadening game support is a feature addition and not a breaking change, because nothing existing stops working. Any middle number above the current one is therefore correct, and the gap leaves room for smaller releases in between.

Everything factual in this document was checked against the source at v2.0.1 rather than recalled. The one item deliberately left open is whether BattleMetrics renders other games from the same template, which cannot be settled without fetching real pages.